

Modelli linguistici e Q-learning: inizializzare la tabella dei valori nell'ambiente MiniGrid-DoorKey

Pietro Fazion
Università di Verona

29 settembre 2026

Sommario

L'ambiente MiniGrid-DoorKey assegna la ricompensa solo quando l'agente raggiunge l'obiettivo finale. Il Q-learning classico impiega molti episodi per registrare il primo successo. Questo documento valuta una singola ipotesi. L'ipotesi prevede l'uso di un modello linguistico per stimare il valore ottimo su un gruppo ridotto di stati. Il sistema inserisce questi valori nella tabella Q prima di iniziare l'addestramento.

L'esperimento su un livello specifico elabora 20 coppie stato-azione relative ai passaggi decisivi. Questa configurazione raggiunge un tasso di successo del 100% con 17 passi medi. La configurazione di controllo senza stime iniziali registra un tasso dello 0%. Il modello gpt-oss-120b presenta un errore medio di 0.0122. Il modello indica l'azione corretta nel 94.1% dei casi. Il trasferimento dell'apprendimento su mappe nuove non produce risultati. Il testo cita i file di log per la verifica dei dati.

Indice

1	Il problema	3
2	Struttura della repository	3
3	L'ambiente e il valore ottimo	3
4	Selezione degli stati	4
4.1	Tipologie di nodi decisivi	4
4.2	Campionamento della progressione	5
5	Interrogazione del modello	5
6	Precisione dei calcoli	6
7	Qualità dell'azione suggerita	6

8	Impatto delle stime iniziali	6
9	Scelta dei parametri	16
10	Limiti dell'approccio	16
11	Sviluppi futuri	16
A	Appendice: ambiente FrozenLake	18

1 Il problema

L'algoritmo Q-learning propaga il valore dall'obiettivo agli stati precedenti [1]. Le definizioni standard di Processo Decisionale di Markov e l'equazione di Bellman restano invariate.

La ricompensa arriva raramente nelle fasi iniziali. La tabella mantiene valori nulli per molto tempo. L'agente non riceve indicazioni per prendere decisioni.

L'esperimento stima il valore V_{opt} con un modello linguistico su pochi stati. Il codice inserisce i valori nella tabella Q iniziale. L'addestramento prosegue con il Q-learning classico. Il processo non genera ricompense aggiuntive. L'obiettivo temporale resta invariato.

I valori iniziali guidano l'agente nei punti decisivi. Questa operazione compensa la mancanza di ricompense immediate.

L'esperimento principale valuta un singolo livello. L'addestramento dura 2000 episodi. Il tasso di apprendimento α vale 0.07. Il fattore di sconto γ vale 0.9. La configurazione adotta 20 coppie stato-azione generate dal modello. Questa configurazione completa la mappa nel 100% dei casi con 17 passi di media. La configurazione senza stime iniziali fallisce in ogni tentativo e consuma 450 passi. I dati risiedono nella cartella `src/output/agents/`. I file di salvataggio memorizzano i valori in formato JSON. La Sezione 8 discute i risultati.

2 Struttura della repository

Il codice sorgente e i dati risiedono nella repository ufficiale del progetto. L'indirizzo web è <https://github.com/ttetrao/llm-guided-rl/tree/main>. La cartella `src/` contiene i moduli operativi. I programmi si avviano da questa posizione. Il comando è `python3-msrc.graph.nome_modulo`. Lo schema si applica a tutti i pacchetti. Il pacchetto `src/graph/` costruisce il grafo del processo decisionale e seleziona gli stati. Il pacchetto `src/llm/` interroga i modelli linguistici. Il pacchetto `src/evaluate/` genera le metriche e i grafici. Il pacchetto `src/agent/` esegue il Q-learning e gestisce i controlli. Il pacchetto `src/env/` contiene gli adattatori per l'ambiente e definisce le fasi. Il pacchetto `src/docs/` raccoglie le istruzioni testuali per i modelli.

Il modulo `src/paths.py` gestisce le destinazioni di salvataggio. I dati finiscono sotto `src/output/`. La cartella `cache` conserva i grafi. La cartella `llm` ospita le risposte dei modelli. La cartella `agents` raccoglie i dati degli addestramenti. Le cartelle `evaluate` contengono i grafici di riepilogo. Il file `src/agent/doorkey_qtable_llminit.py` produce i confronti diretti. I documenti di tesi risiedono in `thesis/`.

3 L'ambiente e il valore ottimo

Il progetto adotta l'ambiente MiniGrid-DoorKey-8x8-v0 [2]. Le regole sono originali. L'agente raccoglie la chiave. L'agente apre la porta con l'azione `toggle`. L'agente raggiun-

ge l'obiettivo fisso sulla mappa. La ricompensa vale 1 solo sul bersaglio. La ricompensa vale 0 in ogni altra cella. Il limite massimo di passi è 450.

Lo stato interno del grafo richiede cinque variabili. Le variabili sono la posizione, la direzione dell'agente, il possesso della chiave e lo stato della porta. La posizione dei muri varia per ogni livello. Queste cinque variabili descrivono la mappa in modo univoco per ogni configurazione.

Lo script `src/graph/mdp_graph.py` genera il grafo. Il calcolo considera sette azioni per ogni stato. Il processo determina la cella di arrivo e la ricompensa. Il grafo contiene 480 nodi per ogni layout. Il bersaglio si trova sempre nella stessa cella. Il calcolo applica il metodo della value iteration e un fattore di sconto 0.99 per determinare il valore ottimo V_{opt} . Questa operazione fornisce il parametro di valutazione per il modello linguistico. Il processo esclude gli stati finali dalle interrogazioni. Il sistema ignora l'azione nulla.

Lo script `src/env/view_wrapper.py` traccia il progresso dell'agente. Lo stato `find_key` indica la ricerca della chiave. Lo stato `open_door` indica il possesso della chiave e la porta chiusa. Lo stato `reach_goal` indica il via libera verso l'obiettivo. L'agente deve rispettare questa sequenza. Due posizioni uguali con stati di avanzamento diversi richiedono azioni diverse.

4 Selezione degli stati

L'interrogazione completa dei 480 nodi richiede molto tempo. Molte celle di passaggio condividono lo stesso valore. Un parametro definisce il numero massimo di interrogazioni. Il file `src/graph/doorkey_states.py` seleziona gli stati utili.

4.1 Tipologie di nodi decisivi

La funzione `extract_bottleneck` dal file `src/graph/doorkey_states.py` isola i nodi decisivi dal grafo. Il processo non richiede l'esecuzione di partite. I nodi si dividono in sei categorie.

1. `on_key`: la cella contiene la chiave. Questa categoria produce 8 nodi.
2. `on_door`: la cella contiene la porta aperta. Questa categoria produce 8 nodi.
3. `adj_door`: l'agente fronteggia la porta chiusa da una cella adiacente. Questa condizione permette di interagire. La categoria produce da 3 a 4 nodi.
4. `door_toggle`: l'azione `toggle` apre la porta con successo. La categoria conta sempre 2 nodi.
5. `key_take`: l'azione di raccolta acquisisce la chiave. La categoria conta da 2 a 8 nodi in base alla posizione dei muri.
6. `goal_entry`: un passo separa l'agente dall'obiettivo. La categoria fornisce da 4 a 8 nodi.

Queste categorie mappano posizioni, condizioni di azione e stati finali. I nodi totali variano tra 30 e 34. Questo numero rappresenta meno del 10% dell'intero grafo.

4.2 Campionamento della progressione

I nodi decisivi marcano i passaggi obbligati. I nodi di progressione fotografano l'evoluzione dell'agente nel tempo. L'agente visita molte celle inutili all'inizio. L'agente si concentra sul percorso ottimale alla fine. La funzione `run_pilot` dal file `src/graph/doorkey_states.py` addestra un agente senza aiuti esterni. I parametri sono $\alpha = 0.3$ e $\gamma = 0.99$. L'esplorazione decade lentamente.

Il programma salva gli stati visitati quando il successo medio tocca soglie specifiche. Le soglie sono 0.3, 0.5, 0.7, 0.8 e 1.0. I salvataggi per i valori bassi includono tutti i percorsi tentati. I salvataggi per i valori alti aggiungono deviazioni casuali. Questo accorgimento impedisce di registrare solo la sequenza perfetta. Il livello di test attiva le soglie agli episodi 93, 115, 137, 150 e 175. Ogni attivazione salva circa 300 nodi.

Alcuni scenari isolati completano il gioco al primo tentativo per puro caso. Tutte le soglie si attivano simultaneamente. Le liste contengono pochissimi nodi. Il codice riporta questi eventi per tracciamento.

La funzione `allocate` dal file `src/graph/doorkey_states.py` riserva il primo spazio per i nodi decisivi. Il programma divide il resto della capienza tra i nodi di progressione calcolando i resti maggiori.

5 Interrogazione del modello

Il programma calcola il valore per l'azione Q a partire dal valore dello stato V . Il testo di istruzione risiede in `src/docs/doorkey/it/prompt.txt`. I file `src/llm/query_gpt.py` e `src/llm/query_gemma.py` gestiscono le chiamate.

Il modello riceve le regole del gioco e la legenda della mappa. Il testo include le definizioni per il calcolo del valore. Il programma invia la mappa corrente e le sei mappe relative alle possibili mosse. Il modello linguistico valuta solo la distanza dall'obiettivo per ogni opzione. L'espressione valutata è γ^d con $\gamma = 0.99$. Il calcolo sfrutta il determinismo del gioco. Il valore esatto di uno stato distante d mosse corrisponde a γ^d . Il modello non adopera stime casuali. Il modello risponde nel formato JSON. Ogni risposta include l'applicazione della regola, il conteggio dei passi e il calcolo numerico. Il programma scarta le risposte anomale o errate.

Il test principale impiega il modello gpt-oss-120b su un livello di base. I file occupano la cartella `src/output/llm/`. Il file di output relativo contiene 500 righe di calcolo. Le risposte valide sono 85 su 99 richieste. Il codice attende 65 secondi tra una chiamata e l'altra. Il sistema ritenta 3 volte in caso di errore.

Un secondo test analizza cinque livelli differenti. Il file di output relativo raccoglie 111 stati validi e 629 righe.

6 Precisione dei calcoli

Lo script `src/evaluate/evaluate_llm.py` calcola le metriche. I risultati riempiono il file `src/output/evaluate/log_valutazione.txt`. L'analisi confronta il valore calcolato dal modello linguistico con il valore reale della mappa.

Risultati sullo scenario di base:

- L'errore medio ammonta a 0.0122. Il bias segna -0.0104 . Il 97.4% delle risposte ha un errore inferiore a 0.05.
- Gli indici di correlazione superano lo 0.9.

I nodi decisivi generano l'errore più alto con 0.0140. La ricerca della chiave risulta la fase più soggetta a imprecisioni.

I test sui cinque livelli restituiscono un errore medio di 0.0153 con un bias di -0.0116 . Il 96.8% si posiziona sotto la soglia di 0.05.

Il bias negativo di -0.01 corrisponde all'errore sistematico di circa un passo. Le Figure 1 e 3 mostrano la dispersione dei punti.

7 Qualità dell'azione suggerita

L'algoritmo valuta la correttezza dell'azione proposta dal modello. La valutazione tollera piccole imprecisioni numeriche causate dal limite dei decimali. Dati per lo scenario di base:

- L'azione suggerita risulta essere la migliore nel 94.1% dei casi. L'azione casuale ottiene il 16.7%.
- La perdita attesa causata da scelte sub-ottimali ammonta a 0.0011.
- La correttezza sui nodi decisivi raggiunge l'82.8%.

Il modello sbaglia i conteggi assoluti dei passi ma identifica l'azione preferibile. Le Figure 5 e 6 riportano i dati per categorie.

L'analisi sui cinque livelli evidenzia l'azione migliore nel 91.9% delle situazioni.

8 Impatto delle stime iniziali

Il codice sorgente per l'agente è `src/agent/doorkey_qtable_llminit.py`. L'agente ragiona su un sistema di coordinate relative all'obiettivo intermedio. L'approccio unifica il comportamento su mappe diverse. Il sistema inserisce per prime le stime calcolate. Le celle rimanenti partono da zero. Il Q-learning procede elaborando le celle in ordine decrescente di valore stimato.

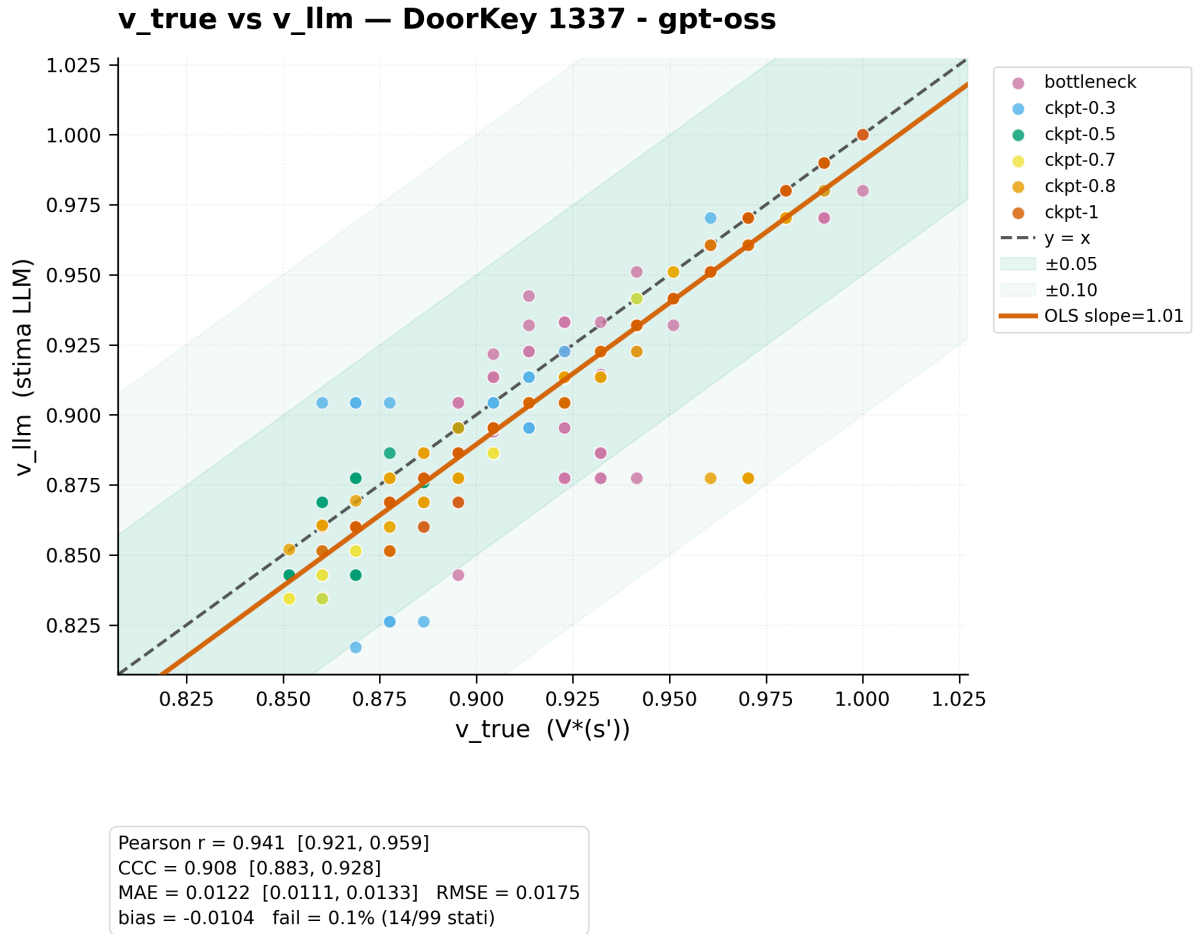


Figura 1: La Figura 1 valuta l'accuratezza del modello linguistico. L'asse orizzontale (X) riporta i valori ottimi attesi. L'asse verticale (Y) espone i valori stimati dal modello. I punti del grafico rappresentano le singole predizioni. Una predizione perfetta cade esattamente sulla diagonale del grafico. I dati formano un agglomerato denso attorno a questa linea. Il modello fornisce stime vicine al valore matematico reale.

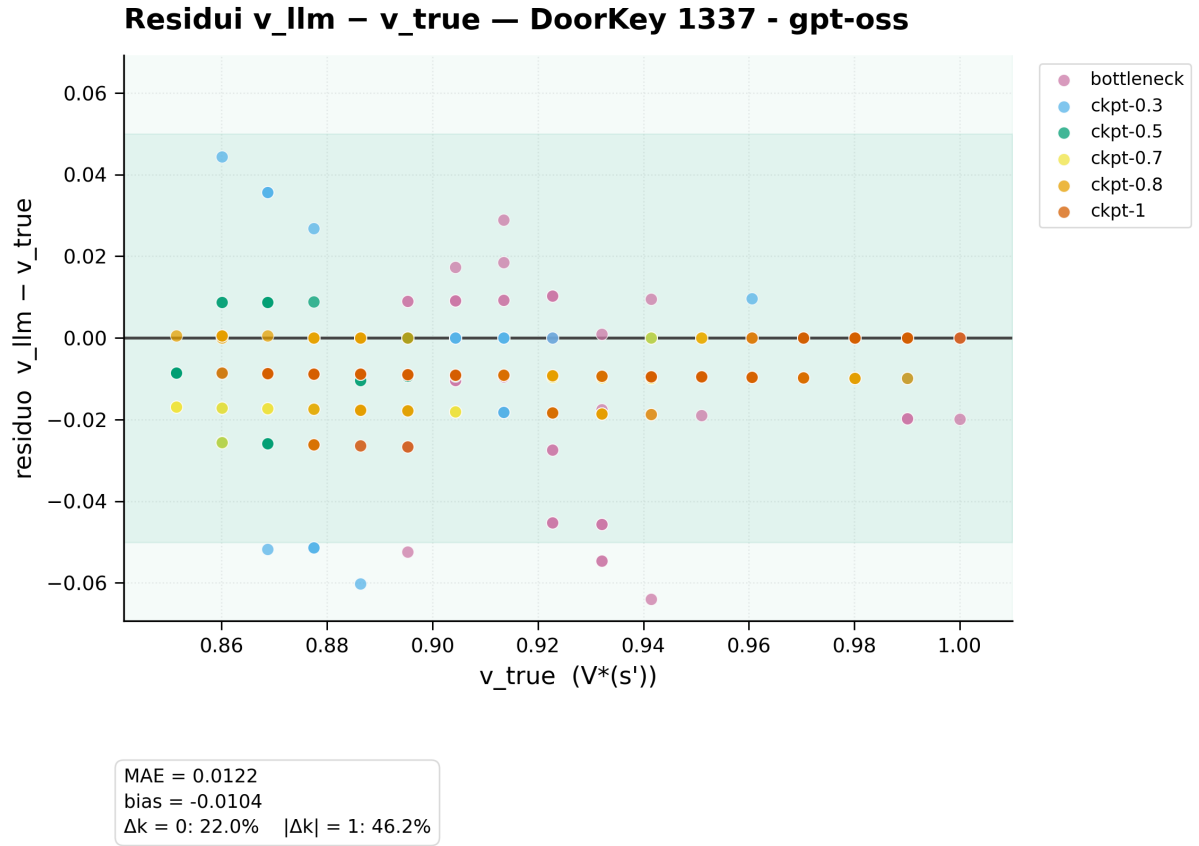


Figura 2: La Figura 2 illustra la distribuzione degli errori del modello. L'asse orizzontale (X) riporta i valori ottimi attesi. L'asse verticale (Y) espone il residuo. Il residuo corrisponde alla differenza matematica tra stima e valore reale. I punti posizionati sulla linea dello zero indicano predizioni perfette. I punti sotto la linea dello zero indicano una sottostima. La maggioranza dei punti si colloca nella parte inferiore del grafico. Il modello genera valutazioni inferiori rispetto ai valori effettivi.

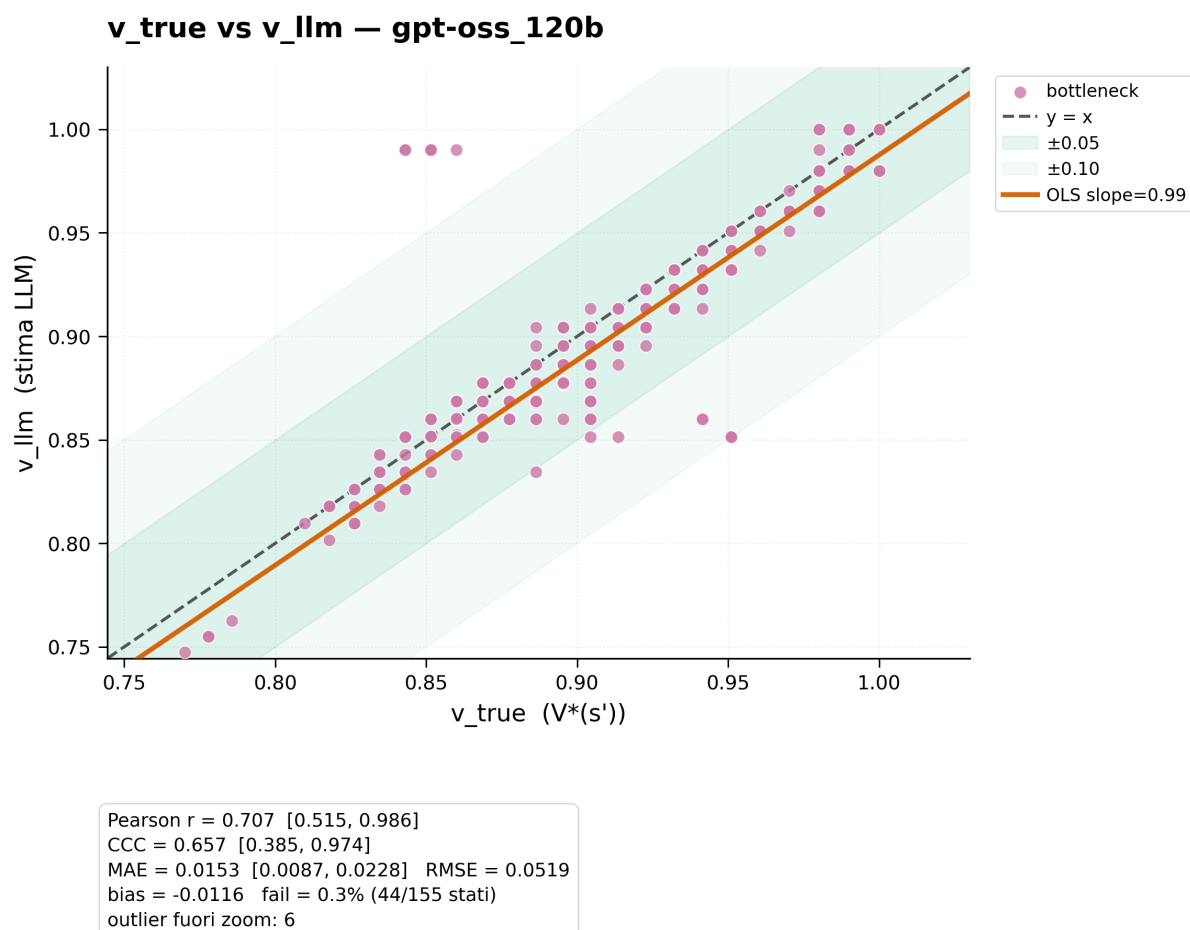


Figura 3: La Figura 3 ripete l'analisi di accuratezza includendo cinque layout di mappa differenti. L'asse orizzontale (X) indica i valori ottimi calcolati. L'asse verticale (Y) indica le stime. I punti mostrano una dispersione maggiore rispetto alla valutazione su singola mappa. La distanza dalla diagonale aumenta. L'introduzione di ostacoli variabili peggiora la precisione delle stime.

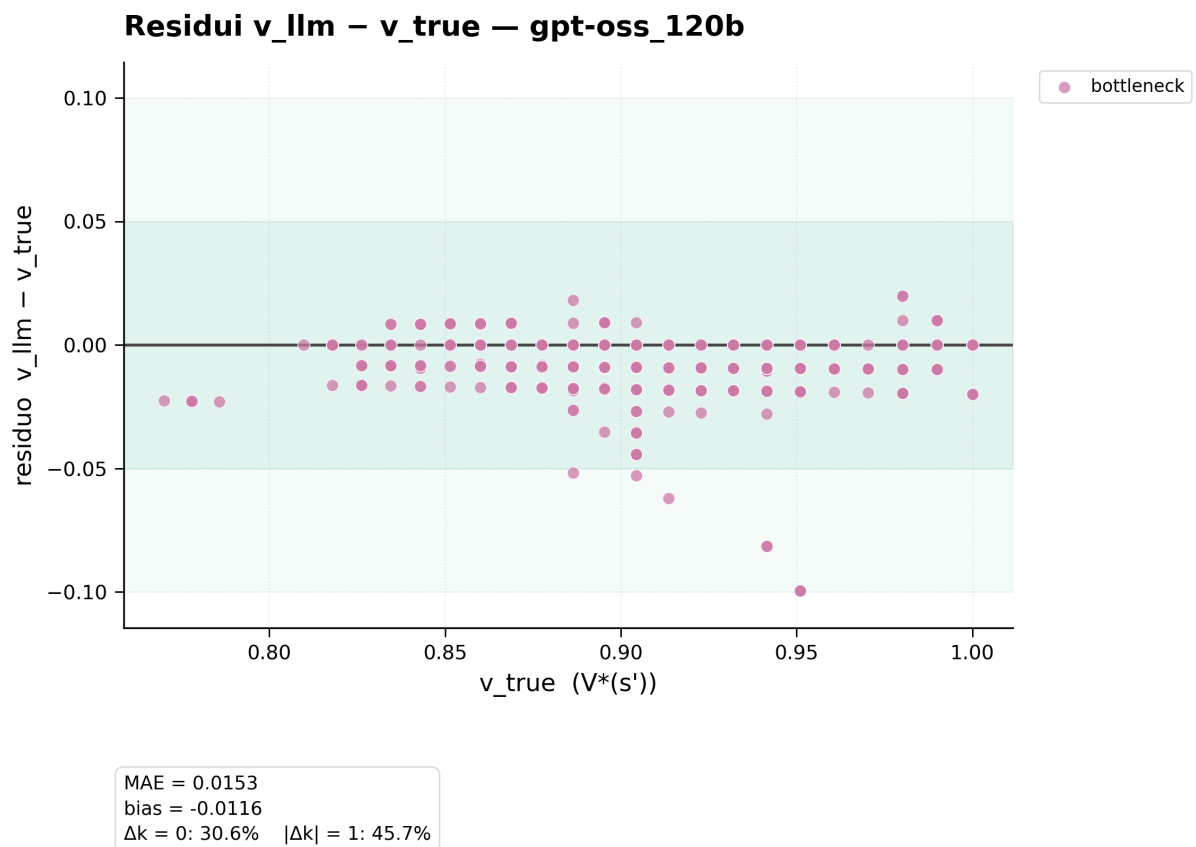


Figura 4: La Figura 4 espande l'analisi dei residui sui cinque layout di mappa. L'asse orizzontale (X) indica i valori attesi. L'asse verticale (Y) quantifica l'errore. La distribuzione dei punti copre un intervallo verticale maggiore. La densità permane sotto la linea dello zero. L'errore sistematico di sottostima si ripete in tutti gli scenari testati.

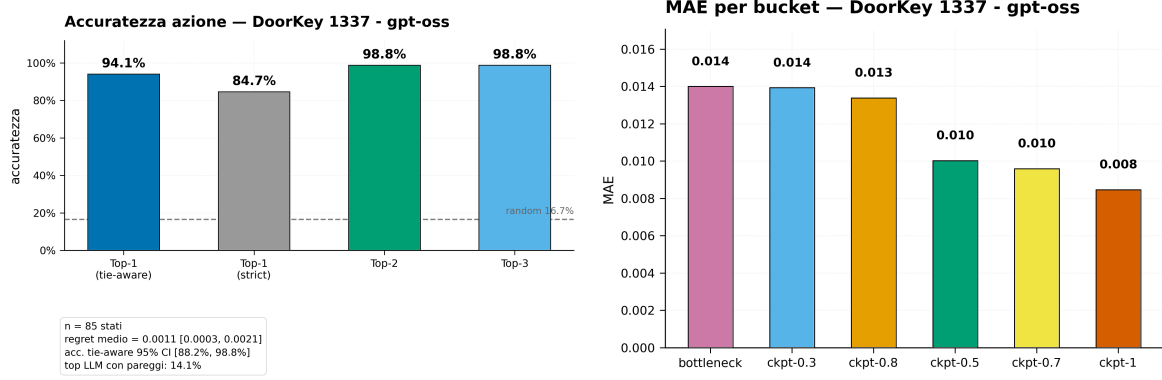


Figura 5: La Figura 5 frammenta i risultati in base alla tipologia di stato valutato. Il primo grafico a barre quantifica le selezioni corrette del modello. Una selezione corretta equivale all'azione migliore possibile per quello specifico stato. Il secondo grafico misura l'errore numerico per categoria. La categoria dei nodi decisivi registra la minor precisione. L'interazione con la porta genera maggiore incertezza nei calcoli del modello.

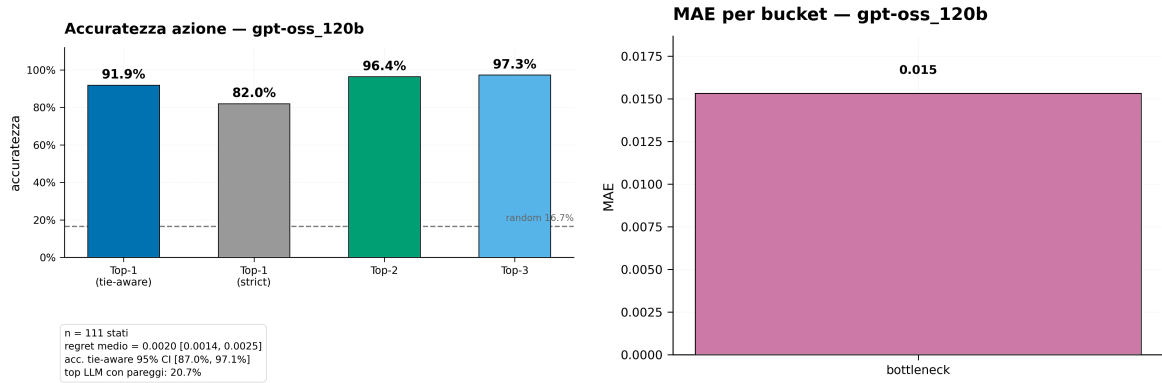


Figura 6: La Figura 6 replica l'analisi categoriale applicata ai cinque scenari estesi. Il grafico sinistro riporta il tasso di successo nell'identificare l'azione ottimale. Il grafico destro quantifica l'errore medio assoluto. Il peggioramento delle metriche coinvolge uniformemente tutte le tipologie di stato. Le interazioni con gli oggetti di gioco si confermano le fasi di maggiore complessità di calcolo.

La funzione `optimal_policy_metrics` dal file `src/agent/doorkey_qtable_llminit.py` valuta il comportamento dell'agente rispetto alla soluzione teorica perfetta.

La prima metrica definisce l'accordo con la politica ottima. Questo parametro conta le volte in cui la decisione dell'agente coincide con l'azione ideale calcolata a priori. Un valore pari a 1 indica una totale aderenza alla strategia perfetta. Un valore basso indica scelte sub-ottimali. Questo numero valuta la precisione delle regole apprese.

La seconda metrica calcola la perdita di valore media. Questo indicatore quantifica il costo degli errori. Ogni mossa sbagliata allontana l'agente dall'obiettivo e abbassa il punteggio atteso. Una perdita media di 0.0067 genera la perdita di una frazione di turno di gioco. Valori prossimi allo zero segnalano traiettorie ottime. Valori alti evidenziano percorsi tortuosi.

Il sistema elabora entrambe le metriche esclusivamente sui nodi esplorati durante le partite effettive.

I test valutano lo scenario di base per 2000 episodi. Il parametro α vale 0.07. Il parametro γ vale 0.9. Il decadimento vale 0.99. Il codice inserisce i 20 nodi decisivi principali.

- L'agente inizializzato completa la mappa in 17 passi con un accordo di 0.585.
- L'agente standard con gli stessi parametri non completa la mappa e impiega 450 passi.
- L'agente standard con parametri intensivi ($\alpha = 0.25$, $\gamma = 0.99$) completa la mappa in 16 passi.

Il controllo senza inizializzazione fallisce con gli stessi parametri base. Le stime innescano la convergenza anticipata dell'algoritmo (Figure 7 e 8).

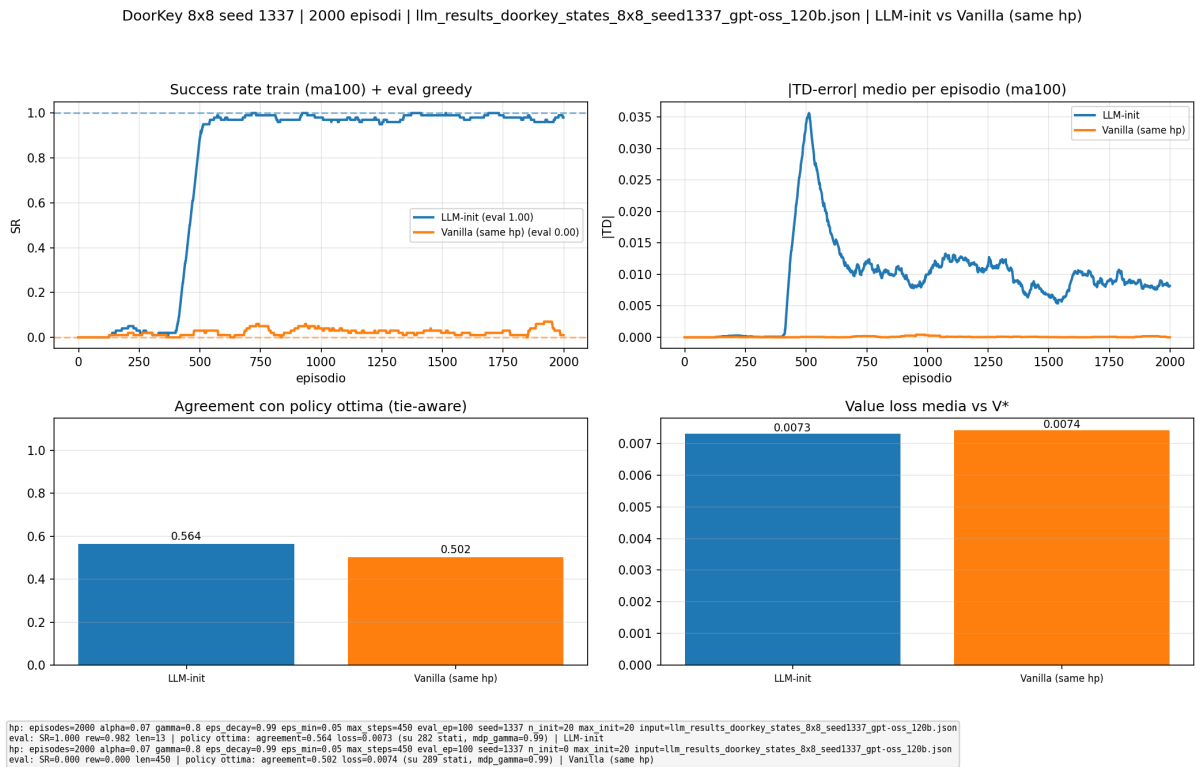


Figura 7: La Figura 7 traccia il progresso dell'apprendimento per rinforzo. L'asse orizzontale (X) numera gli episodi di addestramento progressivi. L'asse verticale (Y) riporta la percentuale di vittorie. La linea superiore rappresenta l'agente dotato di tabella precompilata dal modello linguistico. La linea inferiore rappresenta l'agente basato su tabella vuota. L'agente potenziato raggiunge il 100% di completamento. L'agente di controllo non risolve mai il livello. Le stime innescano la soluzione del problema.

Il secondo ciclo di test esegue l'addestramento su cinque livelli differenti. Le mappe variano la disposizione dei muri. L'addestramento valuta le coordinate relative e prosegue per 4000 turni. I parametri salgono a $\alpha = 0.1$ e $\gamma = 0.8$. Il sistema inserisce 500 stime iniziali. Il codice verifica le prestazioni su un livello già affrontato e su un nuovo livello di test. Il tasso di successo sulle mappe apprese raggiunge il 100% con 23 passi di media. Il tasso di successo sulla mappa nuova si ferma allo 0%. I controlli senza stime registrano risultati

DoorKey 8x8 seed 1337 | 2000 episodi | llm_results_doorkey_states_8x8_seed1337_gpt-oss_120b.json | LLM-init vs Vanilla-std

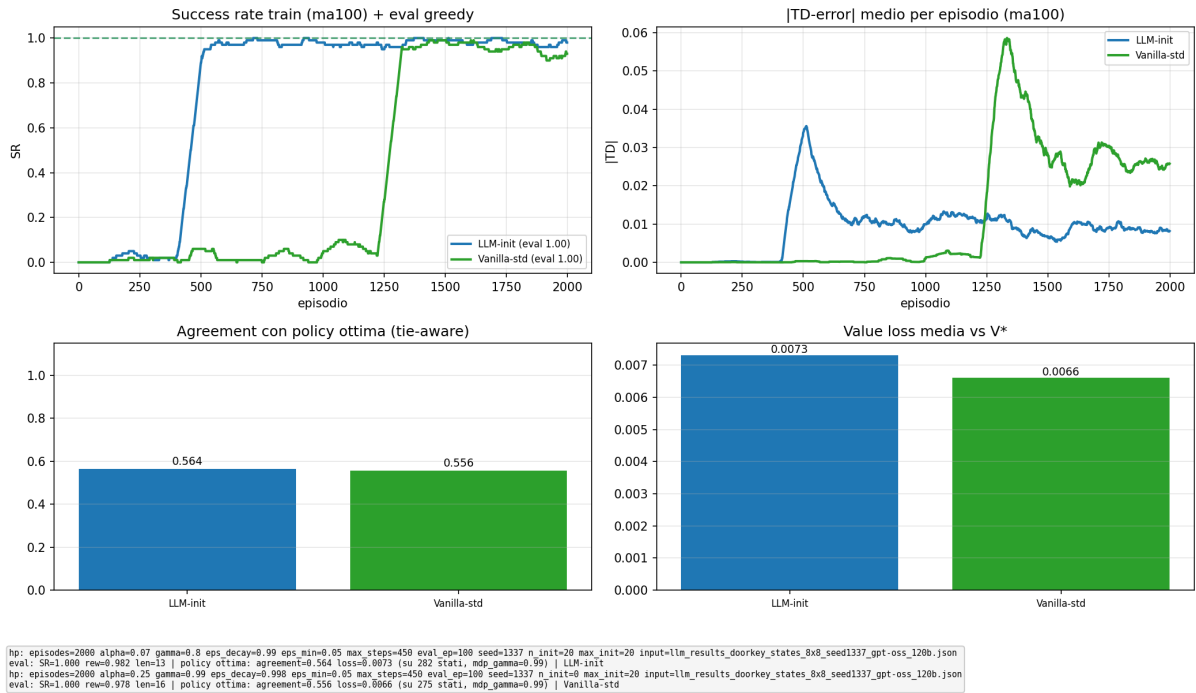


Figura 8: La Figura 8 confronta l'agente precompilato con un agente standard dotato di iperparametri ottimizzati per convergere. L'asse orizzontale (X) numera gli episodi giocati. L'asse verticale (Y) misura le vittorie. Entrambi gli agenti apprendono la strategia corretta. L'agente precompilato richiede un numero inferiore di episodi per completare i primi percorsi. La pendenza iniziale della curva ne certifica la superiorità nelle primissime fasi.

DoorKey 8x8 seed 13507 | 4000 episodi | llm_results_doorkey_states_8x8_seeds13507-19920-32455-54937-69693_gpt-oss_120b.json | LLM-init vs Vanilla (same hp)

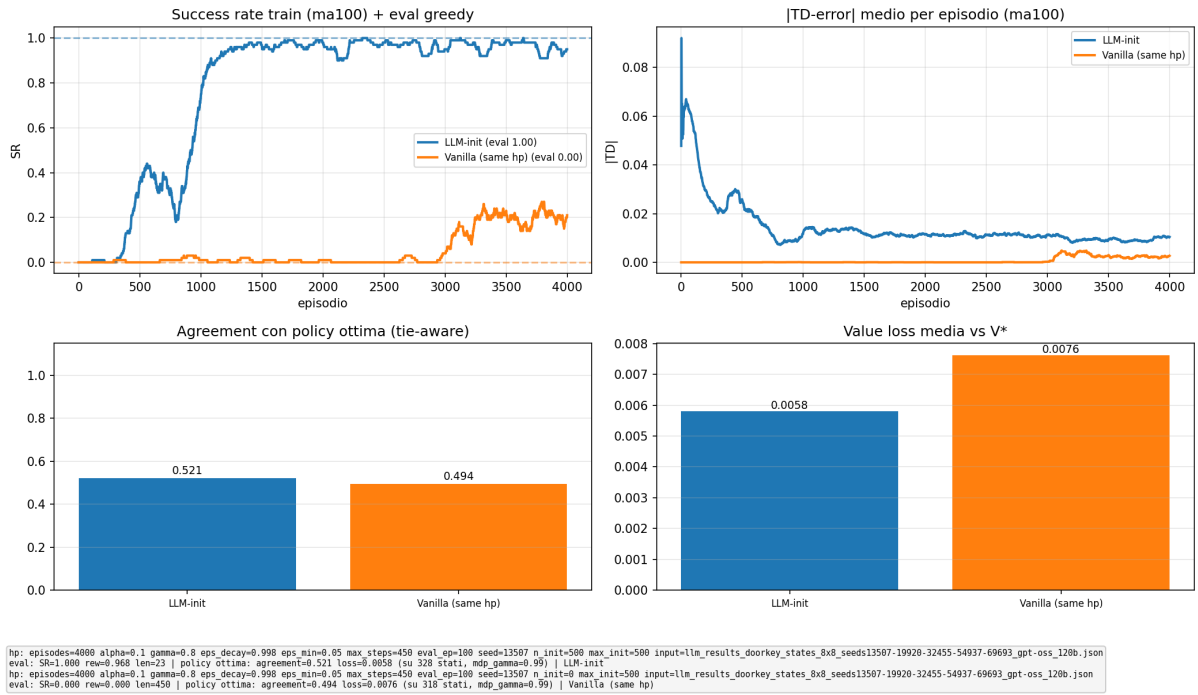


Figura 9: La Figura 9 valuta la capacità di generalizzazione del sistema. Il test esamina il comportamento degli agenti all'interno di un livello escluso dalla fase di addestramento. L'asse orizzontale (X) numera gli episodi. L'asse verticale (Y) registra il tasso di successo. Le curve rimangono ancorate allo zero. Nessun agente supera il livello. L'apprendimento risulta vincolato alla topologia fissa dei muri incontrati in addestramento.

DoorKey 8x8 seed 13507 | 4000 episodi | llm_results_doorkey_states_8x8_seeds13507-19920-32455-54937-69693_gpt-oss_120b.json | LLM-init vs Vanilla-std

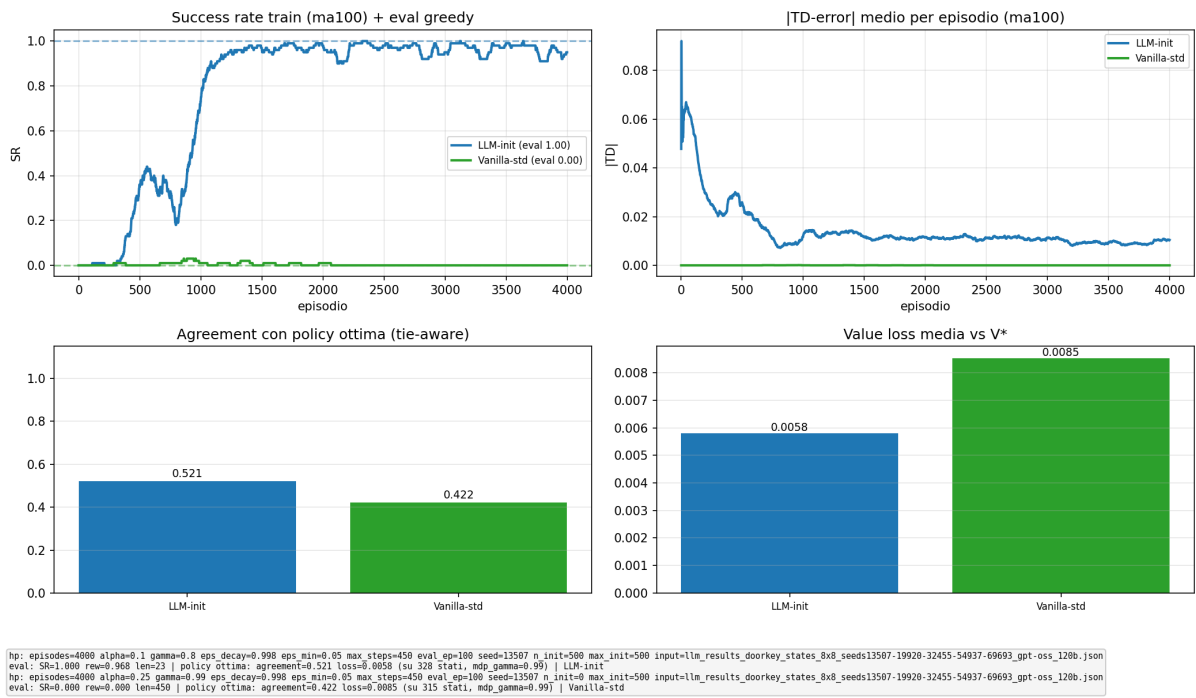


Figura 10: La Figura 10 ribadisce l'incapacità di risolvere livelli inediti introducendo parametri di addestramento aggressivi. L'asse orizzontale (X) numera gli episodi. L'asse verticale (Y) riporta il tasso di completamento. Il grafico mostra il fallimento totale. Le regole apprese sui cinque scenari di base generano azioni inutili o dannose di fronte a una configurazione di muri sconosciuta.

nulli. Cinque layout non consentono al sistema di imparare regole generali esportabili su nuovi livelli (Figure 9 e 10).

I test con quantità variabili di stati mostrano un apprendimento accelerato in presenza di numerosi nodi. Il numero finale di passi converge sempre verso il minimo indispensabile.

9 Scelta dei parametri

La configurazione base adotta $\alpha = 0.07$, $\gamma = 0.9$, limite esplorativo minimo 0.05 e blocco a 450 passi.

- Il parametro γ scende a 0.9. I valori alti appiattiscono la distanza numerica tra due celle vicine. Un valore di 0.9 aumenta lo scarto e facilita la scelta dell'azione migliore.
- Il programma adotta un tasso α ridotto. La tabella parte popolata di valori sensati. Un tasso di aggiornamento massiccio distrugge queste stime prematuramente. I test con $\alpha = 0.9$ mostrano un avvio rapido accompagnato da stabilizzazione ritardata.
- Il limite sui passi consente i vagabondaggi esplorativi nelle prime fasi senza interrompere la partita prematuramente.

10 Limiti dell'approccio

Il problema principale consiste nell'incapacità di risolvere mappe mai viste. Il sistema unifica le coordinate. I muri e gli ostacoli variano per ogni schema. Il modello linguistico richiede molto tempo per calcolare le valutazioni per singolo nodo. La copertura parziale dei nodi funziona bene su mappe contenute. L'impatto si riduce drasticamente su mappe vaste. L'esecuzione dell'algoritmo preliminare su due scenari produce statistiche anomale dettate dal caso.

11 Sviluppi futuri

I risultati sul singolo livello dimostrano l'efficacia del meccanismo. L'agente apprende velocemente rispetto ai modelli standard. L'esperimento su cinque mappe fallisce nel generare regole universali per mappe nuove. Tre punti necessitano di ulteriore sviluppo.

1. **Estensione del set di addestramento.** L'uso di molte decine di livelli permette all'agente di imparare a gestire ostacoli imprevisti.
2. **Modelli dedicati.** Una rete neurale specializzata in stime numeriche riduce i tempi di attesa. La tabella iniziale si popola in tempi inferiori.
3. **Stima in tempo reale.** Il sistema interroga il modello solo durante l'esplorazione di celle nuove. Questa azione evita il calcolo anticipato di tutti gli stati.

I test attuali rappresentano prove di fattibilità eseguite in tiratura singola per ridurre i costi computazionali.

A Appendice: ambiente FrozenLake

L'ambiente **FrozenLake-v1** introduce esiti probabilistici sulle mosse. Il sistema interroga un modello linguistico sui 18 nodi decisivi.

I risultati con il modello **gpt-oss-120b** riportano un errore elevato di 0.1228. Le selezioni corrette si fermano al 55.6%. I modelli faticano a calcolare la media delle probabilità tra esiti incerti tramite ragionamento testuale. Il problema necessita di altre verifiche.

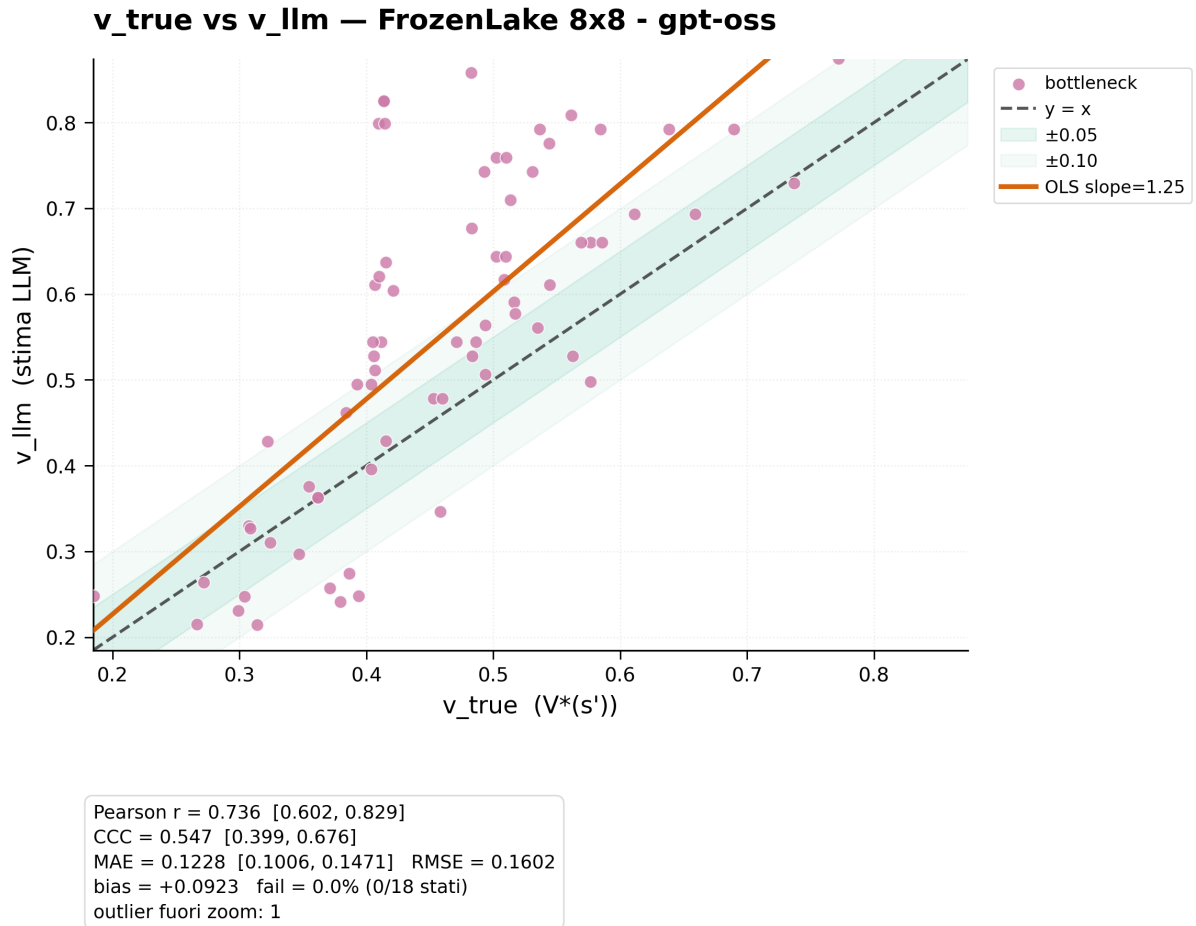


Figura 11: La Figura valuta l'accuratezza del modello **gpt-oss-120b** sull'ambiente **FrozenLake**. L'asse orizzontale (X) riporta i valori ottimi calcolati. L'asse verticale (Y) espone i valori stimati. I punti evidenziano una forte discrepanza rispetto alla diagonale ideale. Il modello genera valutazioni imprecise in presenza di esiti stocastici.

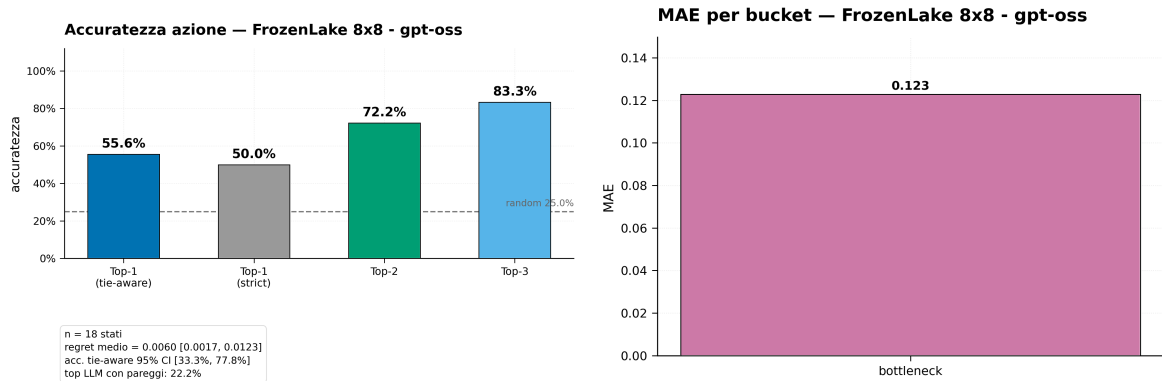


Figura 12: La Figura frammenta i risultati del modello gpt-oss-120b sull'ambiente FrozenLake in base alla tipologia di stato valutato. Il primo grafico a barre quantifica le selezioni corrette. Il secondo grafico misura l'errore numerico. Gli errori risultano distribuiti uniformemente. L'incertezza penalizza l'efficacia del modello su tutti gli scenari.

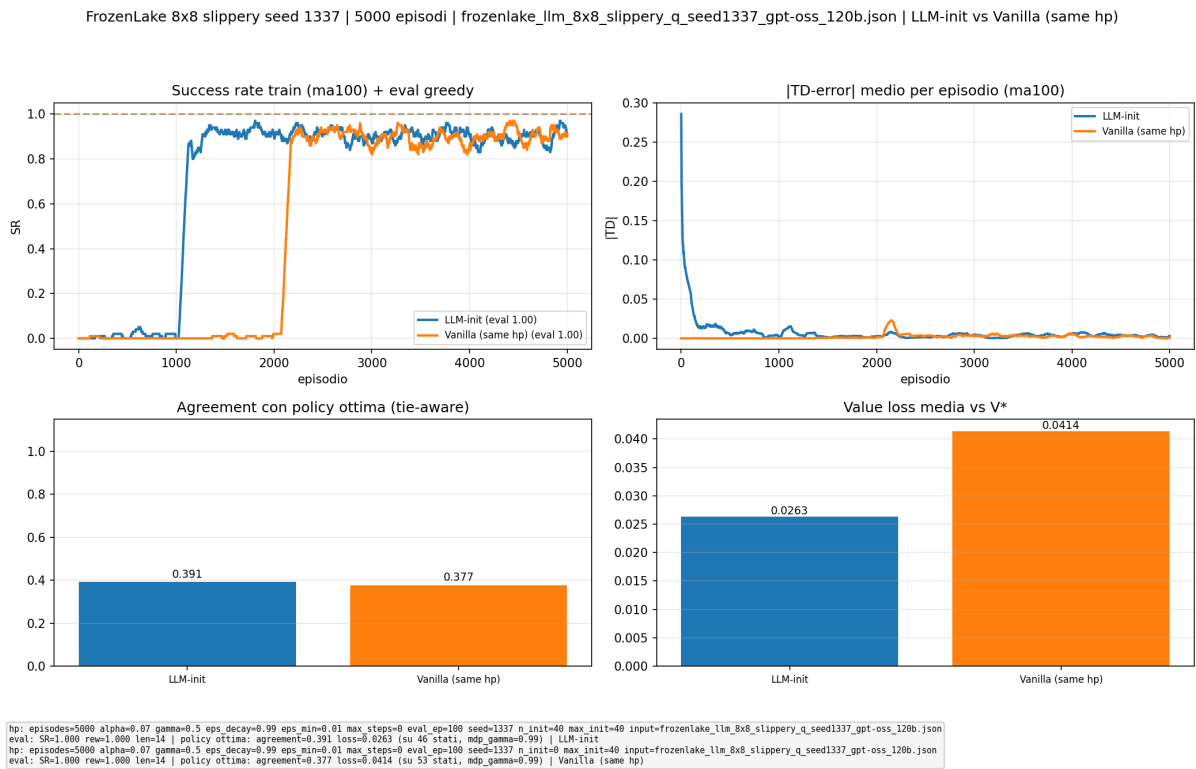


Figura 13: La Figura traccia il progresso dell'apprendimento per rinforzo sull'ambiente FrozenLake. L'asse orizzontale (X) numera gli episodi di addestramento. L'asse verticale (Y) indica il successo. La linea relativa all'agente con stime iniziali e la linea dell'agente standard evidenziano l'impatto dei valori stimati in ambienti probabilistici.

FrozenLake 8x8 slippery seed 1337 | 5000 episodi | frozenlake_llm_8x8_slippery_q_seed1337_gpt-oss_120b.json | LLM-init vs Vanilla-std

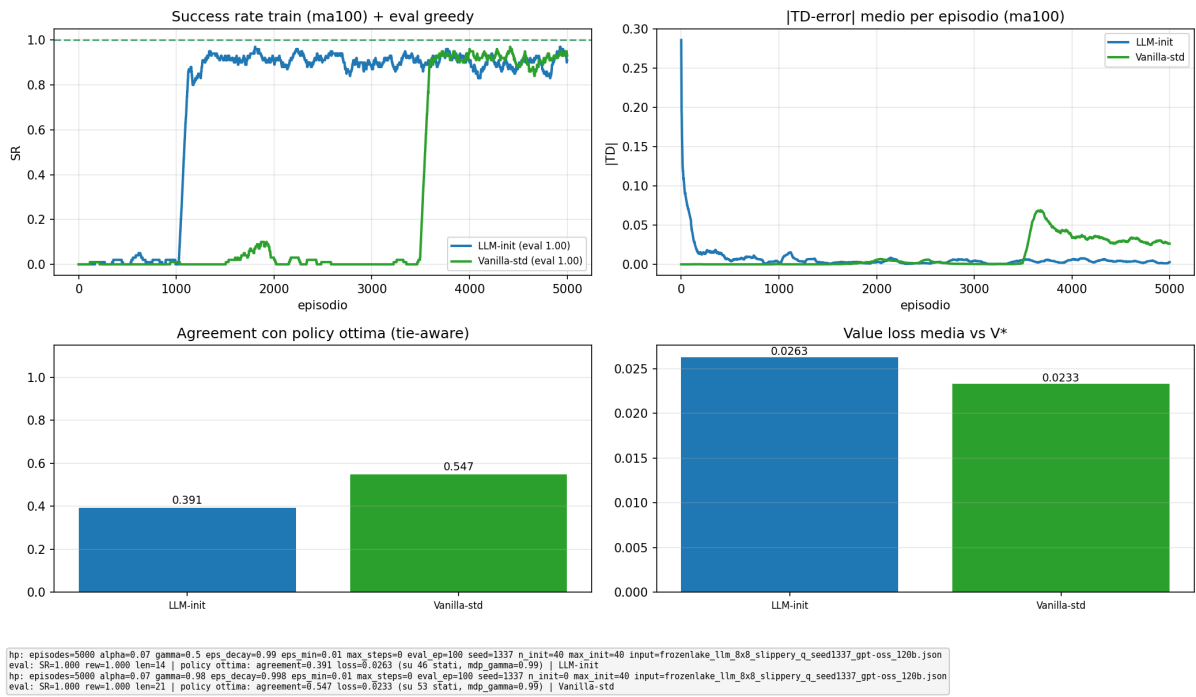


Figura 14: La Figura confronta l'agente precompilato con un agente standard dotato di iperparametri ottimizzati sull'ambiente FrozenLake. Entrambi gli agenti incontrano difficoltà. Le prestazioni indicano l'inadeguatezza dell'attuale metodo di estrazione per scenari non deterministici.

Riferimenti bibliografici

- [1] Sutton, R. S., Barto, A. G. *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [2] Chevalier-Boisvert, M. et al. MiniGrid and MiniWorld. *NeurIPS* 36, 2023.